

20/09/2024

a) dichiarazione PostgreSQL

CREATE TABLE Docente (

matricola CHAR(8), PRIMARY KEY

codice_fiscale CHAR(16), NOT NULL UNIQUE

nome VARCHAR(32),

cognome VARCHAR(32),

data_nascita DATE,

posizione VARCHAR(64) CHECK VALUE IN ('RTD-A', 'RTD-B', 'RTT', 'PA', 'PO', 'PC')
NOT NULL

);

CREATE TABLE Lezione (

- docente CHAR(16), REFERENCES DOCENTE (matricola)

- insegnamento VARCHAR(64),

- data_ora_inizio TIMESTAMP,

data_ora_fine TIMESTAMP,

aula CHAR(3), NOT NULL

PRIMARY KEY (Docente, Insegnamento, data_ora_inizio)

CHECK (data_ora_inizio < data_ora_fine)

);

(i) la chiave primaria sull'attributo matricola di Docente e sulla tupla (docente, insegnamento, data_ora_inizio) di Lezione,

(ii) un vincolo di integrità referenziale Lezione.docente → Docente.matricola,

(iii) un vincolo di dominio posizione ∈ {'RTD-A', 'RTD-B', 'RTT', 'Professore associato', 'Professore ordinario', 'Professore a contratto'}

(iv) l'obbligatorietà dell'attributo aula nella tabella Lezione e dell'attributo posizione nella tabella Docente

(v) un vincolo di superchiave sull'attributo codice_fiscale di Docente

(vi) un vincolo data_ora_inizio < data_ora_fine

b) QUERY

DOCENTE (matricola, codice_fiscale, nome, cognome,
data_nascita, posizione)

LEZIONE (docente, insegnamento, data_ora_inizio, data_ora_fine, aula)

INSEGNAMENTO (codice, titolo, CFU, anno_corso)

- i) Trovare num lezioni x docente e insegnamento, riportando nome e cognome del docente, num di lezioni, data prima e ultima lezione, titolo dell'insegnamento

```
SELECT D.nome, D.cognome, COUNT(*) AS num-lezioni,  
       I.titolo  
       MIN(L.data_ora_inizio), MAX(L.data_ora_fine)  
FROM DOCENTE AS D JOIN LEZIONE L ON  
     (D.matricola = L.docente) JOIN INSEGNAMENTO I  
     ON (L.insegnamento = I.codice)  
GROUP BY D.matricola, I.codice
```

- ii) Trovare TUTTI i docenti con posizione 'PO' che NON hanno effettuato lezioni in aprile e maggio 2023, riportando matricola, CF, nome, cognome e data prima lezione da loro tenuta

```
SELECT D.matricola, D.codice_fiscale, D.nome, D.cognome,  
       MIN(L.data_ora_inizio)  
FROM DOCENTE D JOIN LEZIONE L ON  
     (D.matricola = L.docente)  
WHERE D.posizione = 'Professore Ordinario' AND
```

```

D.matricola NOT IN
(SELECT L1.DOCENTE
FROM LEZIONE L1
WHERE L1.data_ora_inizio >= '01/04/2023 00:00' AND
      L1.data_ora_fine <= '31/05/2023 23:59')
GROUP BY D.matricola;

```

iii) Trovare insegnamenti le cui lezioni nel 2023 sono state tenute da un numero di docenti distinti **MAGGIORE** del num. medio docenti x lezioni dello stesso insegnamento nel 2022

```

CREATE VIEW NUM_DOC AS
(SELECT L.insegnamento,
      EXTRACT (YEAR FROM data_ora_inizio) AS YEAR,
      COUNT (DISTINCT DOCENTE) AS NUM
FROM LEZIONE L
GROUP BY L.insegnamento, YEAR
)

```

EXTRACT (YEAR FROM
data_ora_inizio)

```

SELECT *
FROM NUM_DOC AS N
WHERE N.NUM > (SELECT AVG(NUM)
               FROM NUM_DOC
               WHERE YEAR = '2022')
AND YEAR = '2023';

```

iv) Trovare TUTTI docenti con posizione 'RTD-A' e con data nascita superiore al 01/01/1990. Indicare **indici** x ottimizzare l'esecuzione dell'interrogazione.

```
SELECT *
```

```
FROM DOCENTE
```

```
WHERE posizione = 'RTD-A' AND data_nascita >= '01/01/1990'
```

```
CREATE INDEX idx_docente_posizione_data
```

```
ON DOCENTE (posizione, data_nascita)
```

15/07/2025

a) dichiarazione PostgreSQL

```
CREATE TABLE Auto (  
    targa CHAR(7), PRIMARY KEY  
    num_telaio CHAR(17), NOT NULL UNIQUE  
    modello VARCHAR(32),  
    colore VARCHAR(32),  
    km INTEGER,  
    cilindrata INTEGER,  
    posti INTEGER,  
    proprietario CHAR(16) NOT NULL  
);
```

```
CREATE TABLE Infrazione (  
    codice_avis CHAR(18), PRIMARY KEY  
    • auto CHAR(7), REFERENCES AUTO(targa) NOT NULL  
    • automobilista CHAR(16), NOT NULL  
    • data_ora TIMESTAMP, NOT NULL  
    tipo_infrazione VARCHAR(16) CHECK VALUES IN('lieve', 'Media',  
                                                    'Grave') NOT NULL  
    comune VARCHAR(32),  
    indirizzo VARCHAR(64),  
    multa NUMERIC(5,2), NOT NULL  
    punti_patente INTEGER, NOT NULL  
    pagata BOOLEAN DEFAULT FALSE  
    UNIQUE (automobilista, auto, data_ora)  
    CHECK (( tipo_infrazione = 'lieve' AND punti_patente = 0 )  
           OR ( tipo_infrazione = 'Media' AND punti_patente >= 1  
               AND punti_patente <= 10 ) OR  
           ( tipo_infrazione = 'Grave' AND punti_patente > 10 ) )  
);
```

- i. il vincolo di chiave primaria sull'attributo targa di Auto e codice_avviso di Infrazione,
- ii. un vincolo di integrità referenziale Infrazione.auto à Auto.targa,
- iii. un vincolo di dominio tipo_infrazione ∈ {'Lieve','Media','Grave'}
- iv. una superchiave sull'attributo num_telaio di Auto,
- v. una superchiave composta dagli attributi auto, automobilista, data_ora della tabella Infrazione,
- vi. l'obbligatorietà dell'attributo proprietario nella tabella Auto, e degli attributi tipo_infrazione, multa e punti_patente di Infrazione,
- vii. un vincolo su tipo_infrazione = 'Lieve' allora punti_patente = 0, tipo_infrazione = 'Media' allora punti_patente ∈ [1,10], tipo_infrazione = 'Grave' allora punti_patente > 10
- viii. un valore di default per pagata pari a FALSE.

b) QUERY

```
TRENO ( nm-treno, categoria, partenza, orario_part,
        destinazione, orario_arrivo )
FERMATA ( nm-treno, stazione, orario )
STAZIONE ( nome, comune, indirizzo )
```

- i) Trovare TUTTE le stazioni in cui fermano più treni di categoria 'Regionale' che treni di categoria 'Freccia Rossa' !
 Riportare nomi stazioni .

```
SELECT s.nome,
FROM STAZIONE S JOIN FERMATA F
      ON (s.nome = F.stazione) JOIN
      TRENO T ON (T.nm-treno = F.nm-treno)
WHERE T.categoria = 'Regionale'
GROUP BY stazione,
HAVING COUNT(*) > (SELECT COUNT(*)
                    FROM STAZIONE S1 JOIN FERMATA F1
                    ON (F1.stazione = S1.nome) JOIN TRENO T1
                    ON (T1.nm-treno = T1.nm-treno)
```

```
WHERE T1.categoria='Freccia Rossa'  
AND F1.stazione = s.nome))
```

- ii) Trovare TUTTE le stazioni in cui NON fermano treni di categoria 'Regionale' dopo le 20:00 che partano da 'Torino'. Riportare nome e comune stazioni

```
SELECT s.nome, s.comune  
FROM STAZIONE AS S JOIN FERMATA F  
ON (S.nome = F.stazione)  
WHERE s.nome NOT IN (SELECT F1.stazione  
FROM FERMATA F1 JOIN  
TRENO T1 ON  
(F1.num_treno = T1.num_treno)  
WHERE T1.categoria='Regionale'  
AND F1.orario > 20:00 AND  
T1.partenza='Torino')
```

- iii) Trovare quanti treni partono da 'Verona' il cui num di fermate è superiore al numero medio di fermate fatte dai treni che partono da 'Padova'.

```
CREATE VIEW nfermate AS  
(SELECT t.num_treno, COUNT(*) AS num_fermate,  
t.partenza  
FROM TRENO AS t JOIN FERMATA f  
ON t.num_treno = f.num_treno  
GROUP BY t.num_treno)
```

```

SELECT COUNT(*)
FROM nfermate n
WHERE n.partenza = 'Verona' AND
      n.num-fermate > (SELECT AVG(n1.num-fermate)
                        FROM num-fermate n1
                        WHERE n1.partenza = 'Padova')

```

- iv) Trovare TUTTI i treni di categoria 'Freccia Rossa' che partono da 'Venezia'. Indicare **indice** x ottimizzare l'interrogazione.

```

SELECT *
FROM TRENO T
WHERE T.categoria = 'Freccia Rossa' AND
      T.partenza = 'Venezia'

```

```

CREATE INDEX idx-treno-categoria-partenza
ON TRENO(categoria, partenza)

```

ESERCIZI AGGIUNTIVI

STUDENTE (Matricola, Nome, Data Nascita, Corso Laurea)
APPELLO (CodCorso, Anno Accademico, Data Esame, Aula)
ESITO (CodCorso, Anno Accademico, Matricola, Docente,
Voto Finale, Superato, CFU)

Q1: Trovare Studenti che hanno superato almeno 3 esami da 12 CFU

```
SELECT S.Matricola, S.Nome  
FROM STUDENTE AS S  
      INNER JOIN ESITO AS E  
      ON S.Matricola = E.Matricola  
WHERE E.Superato = TRUE  
      AND E.CFU = 12  
GROUP BY S.Matricola, S.Nome  
HAVING COUNT(*) >= 3
```

Q2: Trovare studenti che hanno preso 30 sia in BdD in aula T.01, sia in Programmazione in aula Lab1

```
SELECT S.Matricola, S.Nome  
FROM STUDENTE AS S  
      INNER JOIN ESITO AS E  
      ON S.Matricola = E.Matricola  
      INNER JOIN APPELLO AS A  
      ON E.CodCorso = A.CodCorso  
      AND E.Anno Accademico = A.Anno Accademico
```

```

WHERE E. VotoFinale = 30
AND E. CodCorso = 'Basi di Dati'
AND A. Aula = 'T.01'
AND S. Matricola IN
    (SELECT E1. Matricola
     FROM ESITO AS E1
     INNER JOIN APPELO AS A1
     ON E1.CodCorso = A1. CodCorso
     AND E1. AnnoAccademico = A1. AnnoAccademico
     WHERE E1. VotoFinale = 30
     AND E1. CodCorso = 'Programmazione'
     AND A1. Aula = 'lab1'
    );

```

Q3: Trovare gli studenti che NON hanno sostenuto esami in corsi per i quali si sono tenuti appelli in almeno 2 A.A. diversi

```

SELECT Matricola, Nome
FROM Studente
WHERE Matricola NOT IN
    ( SELECT E. Matricola
      FROM APPELO AS A
      INNER JOIN APPELO AS B
      ON A. CodCorso = B. CodCorso
      AND A. AnnoAccademico <> B. AnnoAccademico
      INNER JOIN ESITO AS E
      ON E. CodCorso = A. CodCorso
    );

```

```

SELECT S. Matricola, S. nome
FROM studente AS S
WHERE NOT EXISTS
(
  SELECT *
  FROM APPELO AS A
  INNER JOIN APPELO AS B
  ON A. CodCorso = B. CodCorso
  AND A. AnnoAccademico <> B. AnnoAccademico
  INNER JOIN ESITO AS E
  ON E. CodCorso = A. CodCorso
  AND E. Matricola = S. Matricola
);

```

Q4: X ogni corso di laurea, estrarre la studente che ha acquisito il maggior num. tot. di CFU, ordinando il risultato in modo decrescente

```

CREATE VIEW Totali AS
SELECT Matricola, SUM(CFU) AS CFUTotali
FROM ESITO
WHERE Superato = TRUE
GROUP BY Matricola;

```

```

SELECT S. CorsoLaurea, S. Matricola, S. Nome, T. CFUTotali
FROM studente AS S
    INNER JOIN Totali AS T
    ON S. Matricola = T. Matricola
WHERE T. CFUTotali =
    (SELECT MAX (T1.CFUTotali)
      FROM studente AS S1
        INNER JOIN Totali AS T1
        ON S1. Matricola = T1. Matricola
      WHERE S1. CorsoLaurea = S. CorsoLaurea )
ORDER BY T. CFUTotali DESC;

```

Q5: Trovare Cod. Corso/i che ha registrato il maggior numero di esiti falliti (superato = FALSE) in assoluto tra tutti gli A.A.

```

CREATE VIEW FallimentiPerCorso AS
SELECT CodCorso, COUNT (*) AS NumFallimenti
FROM ESITO
WHERE Superato = FALSE
GROUP BY CodCorso;

```

```

SELECT CodCorso
FROM FallimentiPerCorso
WHERE NumFallimenti = (SELECT MAX (NumFallimenti)
                       FROM FallimentiPerCorso );

```

AUTO (targa, marca, modello, posti, cilindrata)

NOLEGGIO (targa, cliente, inizio, fine^{*})

CLIENTE (nPatente, cognome, nome, paeseProvenienza,
nInfrazioni)

Q: Determinare marche d'auto che hanno il maggior numero di noleggi conclusi. Visualizzare marca, num TOT auto di quella marca, num noleggi con auto di quella marca utilizzata

CREATE VIEW NoleggiTerminatiPerMarca AS

SELECT a.marca, COUNT(DISTINCT a.targa) AS num_auto,
COUNT(*) AS Noleggi

FROM NOLEGGIO AS n JOIN AUTO a

ON n.targa = a.targa

WHERE n.fine IS NOT NULL

GROUP BY a.marca;

SELECT marca, numAuto, noleggi

FROM NoleggiTerminatiPerMarca

WHERE noleggi IN (SELECT MAX(noleggi)

FROM NoleggiTerminatiPerMarca)

UTENTE (cf, nome, cognome, telefono, dataIscrizione, stato)
 PRESTITO (idRisorsa, idBiblioteca, idUtente, dataInizio, Durata)
 RISORSA (id, biblioteca, titolo, tipo, stato)

↓
 TIMESTAMP
 ↓
 INTERVAL

①: Trovare per ogni utente che abbia fatto un numero di prestiti conclusi alla data odierna **SUPERIORI** alla **MEDIA** di tutti i prestiti conclusi, il num di prestiti terminati e la loro durata TOT.
 Nel risultato riportare anche CF utente e conteggi richiesti.

```

CREATE VIEW prestitiPerUtente AS
  select U.codiceFiscale AS idUtente, COUNT(*) AS nPrestiti,
         SUM(durata) AS durata
FROM UTENTE U JOIN PRESTITO P
  ON U.CodiceFiscale = P.idUtente
WHERE durata IS NOT NULL
      AND (P.data_inizio + durata)::date < CURRENT-DATE
GROUP BY U.CodiceFiscale;

```

```

SELECT idUtente, nPrestiti, durata
FROM prestitiPerUtente p
WHERE (nPrestiti) > ANY (SELECT AVG(nPrestiti)
                        FROM prestitiPerUtente);

```